

CAHIER DE NOTES

Projet I.A. — Morpion à cases bloquées

Étude comparative de MinMax et du Q-learning

sur un morpion à cases bloquées

Document de travail finalisé pour modification et enrichissement

Étudiant 1	NKEMLA LIENOU Steve Frede
Étudiant 2	RICHETER Marie
Projet	Morpion à cases bloquées
Algorithmes étudiés	Q-learning et MinMax
Groupe / Date	Groupe 3 / 06/05/2026

Institut Supérieur de l'Électronique et du Numérique

SOMMAIRE

Organisation du cahier de notes

Résumé exécutif

1. Introduction

2. Présentation du jeu étudié

2.1 Morpion classique

2.2 Innovation apportée

2.3 Remarque du professeur et évolution de la réflexion

3. Problématique et objectifs

3.1 Objectif général

3.2 Objectifs spécifiques

4. Démarche scientifique adoptée

5. Solution 1 : Q-learning

5.1 Principe théorique

5.2 Adaptation au projet

5.3 Choix d'implémentation

5.4 Premier résultat observé

6. Solution 2 : MinMax

6.1 Principe théorique

6.2 Adaptation au projet

6.3 Optimisations apportées

6.4 Limite observée

7. Architecture et implémentation

8. Expérimentations

8.1 Protocole général

8.2 Expériences déjà réalisées

8.3 Tableau de suivi des expériences à compléter

9. Résultats obtenus

9.1 Résultat quantitatif déjà validé

9.2 Résultats qualitatifs

10. Analyse comparative

10.1 Impact des cases bloquées

10.2 Q-learning face à MinMax

10.3 Ce que nous retenons

11. Limites du projet

12. Conclusion et ouverture

13. Annexes

13.1 Répartition du travail

13.2 Éléments à ajouter avant rendu final

Résumé exécutif

Dans ce projet, notre binôme a étudié deux approches d'intelligence artificielle sur une version enrichie du morpion : le Q-learning, qui apprend progressivement par l'expérience, et MinMax, qui raisonne à partir d'un arbre de décisions. L'originalité du travail repose sur l'ajout de cases bloquées, qui modifient directement l'espace stratégique du jeu et rendent la comparaison plus intéressante qu'un morpion classique.

Au fil de la réalisation, nous avons constaté qu'un nombre élevé de cases bloquées sur une petite grille conduit trop facilement à des plateaux fermés et donc à des matchs nuls. À partir d'une remarque du professeur, nous avons donc amélioré la conception du jeu en adaptant la taille du plateau au nombre de cases bloquées, afin de conserver un niveau de jeu cohérent et des lignes gagnantes encore accessibles.

Sur le plan technique, le moteur du jeu a été stabilisé, l'agent Q-learning a été entraîné et sauvegardé dans une table Q, puis MinMax a été optimisé grâce à une profondeur limitée, une heuristique d'évaluation et un élagage alpha-bêta. Les premiers résultats montrent que l'apprentissage fonctionne, tout en mettant en évidence les limites de MinMax lorsque la taille de la grille augmente.

Idée centrale du projet

Nous n'avons pas seulement modifié un jeu simple ; nous avons étudié comment une contrainte structurelle, ici les cases bloquées, influence la stratégie de jeu et l'adaptation de deux approches d'intelligence artificielle de nature très différente.

1. Introduction

Le projet d'intelligence artificielle demandé en ISEN2 a pour but de mettre en œuvre concrètement des notions vues en cours à travers une réalisation en binôme. Le cadre de travail nous impose de comprendre et d'expliquer le fonctionnement interne des algorithmes utilisés, sans nous reposer sur des bibliothèques qui masqueraient la logique réelle de l'IA.

Dans ce contexte, nous avons choisi de travailler sur le morpion, car il s'agit d'un jeu simple à décrire mais riche dès que l'on y ajoute des contraintes. Notre version du projet introduit des cases bloquées, puis compare deux approches d'IA : Q-learning et MinMax. Ce choix nous permet d'opposer une logique d'apprentissage progressif à une logique de recherche de coup optimal.

Le sujet est particulièrement intéressant car il combine trois dimensions complémentaires : la modélisation d'un jeu, l'implémentation d'algorithmes d'IA explicables, et l'analyse expérimentale de leurs performances dans un environnement modifié.

2. Présentation du jeu étudié

2.1 Morpion classique

Le morpion classique oppose deux joueurs sur une grille de taille réduite. Chaque joueur place à tour de rôle un symbole sur une case libre. Le premier qui aligne tous ses symboles sur une ligne, une colonne ou une diagonale gagne la partie. Si toutes les cases sont occupées sans alignement complet, la partie se termine par un match nul.

2.2 Innovation apportée

Notre version enrichit ce jeu en introduisant des cases bloquées, représentées par le symbole #. Ces cases ne peuvent être utilisées par aucun joueur. Leur présence modifie immédiatement les trajectoires de victoire possibles, réduit certaines lignes gagnantes et impose une adaptation de la stratégie des joueurs comme des IA.

2.3 Remarque du professeur et évolution de la réflexion

Au départ, l'idée consistait simplement à ajouter un nombre variable de cases bloquées sur une grille standard. Cependant, une remarque du professeur nous a amenés à approfondir la réflexion : si l'on bloque trop de cases sur une grille trop petite, la partie risque de devenir trivialement nulle, car l'espace stratégique devient insuffisant.

Cette observation a modifié notre conception. Nous avons donc choisi d'éviter les configurations trop fermées et d'adapter dynamiquement la taille de la grille selon le nombre de cases bloquées. Ainsi, le projet ne se limite plus à un morpion modifié ; il devient une étude sur l'impact d'une contrainte structurelle sur le jeu et sur les algorithmes d'IA.

Logique retenue

Peu de cases bloquées : grille 3x3. Blocage plus important : grille 4x4, voire plus grande si nécessaire. L'objectif est de conserver assez de lignes ouvertes pour maintenir un intérêt stratégique et limiter les nuls automatiques.

3. Problématique et objectifs

La problématique générale du projet est la suivante : comment les cases bloquées modifient-elles la stratégie de jeu dans un morpion, et comment deux approches différentes, MinMax et Q-learning, s'adaptent-elles à cette contrainte ?

3.1 Objectif général

L'objectif général est de concevoir un morpion à cases bloquées, d'implémenter deux IA capables d'y jouer, puis d'analyser leur comportement et leurs limites dans ce nouvel environnement.

3.2 Objectifs spécifiques

- Concevoir un moteur de jeu stable et modulaire ;
- Ajouter une gestion cohérente des cases bloquées ;
- Implémenter un agent Q-learning et l'entraîner ;
- Implémenter un MinMax exploitable sur différentes tailles de plateau ;
- Comparer qualitativement et quantitativement les deux approches ;
- Mettre en évidence l'impact des cases bloquées sur la stratégie et les performances.

4. Démarche scientifique adoptée

Notre démarche s'est appuyée sur une logique proche de la méthode scientifique présentée en cours : observation d'un problème, formulation d'une hypothèse, mise en œuvre de solutions, puis analyse des résultats obtenus.

L'observation de départ était simple : un morpion classique constitue un cadre de test accessible, mais il devient plus intéressant si l'on modifie sa structure. L'hypothèse initiale était que l'ajout de cases bloquées allait perturber l'équilibre habituel du jeu et différencier plus nettement le comportement des algorithmes.

Nous avons ensuite retenu deux solutions de nature différente. Le Q-learning représente une approche d'apprentissage par renforcement, tandis que MinMax correspond à une approche de recherche arborescente avec hypothèse d'adversaire optimal. Le choix de ces deux méthodes permet une comparaison conceptuelle claire entre apprendre et calculer.

Enfin, nous avons analysé les premiers résultats en termes de victoires, défaites, nuls et temps de décision, tout en identifiant les limites du modèle et les améliorations encore possibles.

5. Solution 1 : Q-learning

5.1 Principe théorique

Le Q-learning est une méthode de renforcement d'apprentissage dans laquelle un agent découvre progressivement de bonnes décisions en interagissant avec son environnement. Dans ce cadre, l'agent observe un état, choisit une action, reçoit une récompense, puis met à jour sa connaissance sous la forme d'une table Q associant des couples état-action à une valeur estimée.

5.2 Adaptation au projet

Dans notre projet, l'état correspond à la configuration actuelle du plateau, l'action à la case choisie par l'agent, et la récompense à l'issue de la partie ou à l'intérêt local du coup joué. Une victoire donne une récompense positive, une défaite une récompense négative, et un match nul une petite récompense intermédiaire.

5.3 Choix d'implémentation

Nous avons implémenté l'agent dans un fichier dédié, avec une table Q mémorisée sous forme de dictionnaire Python. L'agent utilise les paramètres classiques alpha, gamma et epsilon. L'entraînement s'effectue contre un adversaire simple jouant aléatoirement, ce qui permet à l'agent d'apprendre progressivement une politique de jeu. La mémoire acquise est ensuite sauvegardée dans le fichier *q_table.pkl* afin d'être réutilisée dans le jeu principal sans réentraîner l'agent à chaque lancement.

5.4 Premier résultat observé

Sur une campagne d'entraînement de 10000 épisodes, l'agent a obtenu 4883 victoires, 938 défaites et 4179 matchs nuls, soit respectivement 48.8 %, 9.4 % et 41.8 %. Ce résultat montre que l'agent apprend réellement et qu'il perd nettement moins souvent qu'il ne gagne ou ne neutralise la partie.

6. Solution 2 : MinMax

6.1 Principe théorique

MinMax repose sur une logique d'exploration d'arbre de jeu. Le joueur courant cherche à maximiser son gain futur, tandis que l'adversaire est supposé jouer le meilleur coup possible pour le minimiser. L'algorithme raisonne donc à partir d'anticipations successives : mon meilleur coup dépend du meilleur coup de l'adversaire, qui lui-même dépend de ma meilleure réponse, et ainsi de suite.

6.2 Adaptation au projet

Dans notre projet, MinMax est utilisé comme seconde IA afin de comparer une approche fondée sur le calcul avec une approche fondée sur l'apprentissage. L'algorithme doit prendre en compte les cases bloquées, les coups encore possibles et l'état du plateau pour choisir le coup jugé le plus favorable.

6.3 Optimisations apportées

Une première version de MinMax devenait trop lente dès que la taille de la grille augmentait. Nous l'avons donc optimisée avec trois leviers : une profondeur maximale de recherche, une fonction heuristique d'évaluation du plateau, et un élagage alpha-bêta. Cette optimisation réduit fortement le nombre de branches explorées et permet d'obtenir des temps de réponse plus raisonnables sur les grilles 4x4.

6.4 Limite observée

Même optimisé, MinMax reste sensible à l'explosion combinatoire lorsque la taille du plateau augmente. Cette difficulté constitue en elle-même un résultat intéressant, car elle montre qu'une IA de calcul exact ou semi-exact devient rapidement coûteuse dans des environnements plus riches.

7. Architecture et implémentation

Le projet a été structuré en plusieurs fichiers afin de séparer clairement le moteur du jeu, les deux IA et les scripts d'expérimentation. Cette organisation facilite la maintenance, la lisibilité et la présentation du travail.

Fichier	Rôle principal
morpion.py	moteur du jeu, grille, cases bloquées, règles, fin de partie
qlearning.py	définition de l'agent Q-learning et gestion de la table Q
entrainement.py	boucle d'apprentissage contre adversaire aléatoire
minMax.py	algorithme MinMax optimisé avec profondeur et alpha-bêta
evaluation.py	comparaison automatique entre les deux IA
main.py	menu principal et modes de jeu humain / IA
interface.py	Interface graphique du projet : page d'accueil, modes de jeu, entraînement et comparaison des IA

Sur le plan du code, le projet a également évolué de manière importante : amélioration de l'affichage, ajout des coups possibles, représentation des états pour l'IA, sauvegarde de l'apprentissage, optimisation du calcul de MinMax et cohérence entre la taille de la grille et le niveau de blocage.

8. Expérimentations

Les expérimentations ont été menées pour répondre à trois questions principales : le Q-learning apprend-il réellement ? MinMax reste-t-il exploitable lorsque le plateau grandit ? Les cases bloquées modifient-elles suffisamment le jeu pour justifier l'innovation proposée ?

8.1 Protocole général

Le protocole retenu consiste à faire varier le nombre de cases bloquées, la taille du plateau, le type d'adversaire et l'algorithme utilisé. Les résultats observés sont ensuite analysés en termes de victoires, défaites, nuls, qualité de jeu et temps de réponse.

8.2 Expériences déjà réalisées

- Entraînement du Q-learning sur 10 000 épisodes ;
- Tests manuels du moteur du jeu en mode humain contre humain ;
- Tests du mode humain contre Q-learning ;
- Tests du mode humain contre MinMax ;
- Observation des effets d'une augmentation du nombre de cases bloquées ;
- Observation des gains de temps obtenus après optimisation de MinMax.

8.3 Tableau de suivi des expériences

Expérience	Paramètres	Résultats	Observation	Statut
Q-learning — 1 case bloquée	10 000 épisodes	4883 / 938 / 4179	apprentissage visible	Fait
Q-learning — 0 case bloquée	10 000 épisodes	8939 / 416 / 645	apprentissage visible	Fait
Q-learning vs MinMax	100 épisodes	0 / 0 / 100	égalité	Fait
Impact de 2 cases bloquées (Q-learning)	grille 4x4 (10000 épisodes)	4209 / 510 / 5281	jeu moins fermé que 3x3	En cours

9. Résultats obtenus

9.1 Résultat quantitatif déjà validé

Campagne	Victoires	Défaites	Nuls
Q-learning — 10 000 épisodes	4883 (48.8 %)	938 (9.4 %)	4179 (41.8 %)

Ces résultats montrent que l'agent apprend une politique de jeu cohérente. Le nombre de défaites reste faible par rapport au nombre de victoires et de matchs nuls, ce qui indique que le système mémorise progressivement des choix efficaces.

9.2 Résultats qualitatifs

- Les cases bloquées modifient clairement la stratégie de placement ;
- Sur une petite grille, un blocage trop fort conduit rapidement à des plateaux stériles ;
- L'adaptation dynamique de la taille du plateau rend le jeu plus pertinent ;
- MinMax devient plus coûteux lorsque la taille augmente, même après optimisation ;
- Q-learning gagne en intérêt lorsque l'espace de jeu devient moins trivial.

10. Analyse comparative

10.1 Impact des cases bloquées

Les cases bloquées ne sont pas un simple ajout visuel. Elles transforment réellement le jeu, car elles réduisent le nombre de lignes gagnantes accessibles, perturbent les schémas habituels d'alignement et forcent les joueurs comme les IA à revoir leurs priorités. Cette contrainte structurelle justifie pleinement l'intérêt du projet.

10.2 Q-learning face à MinMax

Le Q-learning présente l'avantage d'apprendre progressivement à partir de l'expérience, ce qui le rend particulièrement intéressant dans un environnement modifié. En revanche, sa qualité dépend du temps d'entraînement, du choix des récompenses et du type d'adversaire rencontré pendant l'apprentissage.

MinMax, à l'inverse, n'apprend pas. Il raisonne à partir de l'état courant et tente de calculer le meilleur coup possible. Son comportement est donc plus déterministe et souvent très solide sur les petites grilles. Toutefois, son coût de calcul augmente fortement avec la taille du plateau, ce qui limite sa souplesse lorsque le jeu devient plus complexe.

10.3 Ce que nous retenons

La comparaison montre qu'aucune approche n'est universellement supérieure. MinMax est très pertinent pour les petits espaces de recherche, tandis que le Q-learning devient plus intéressant dès que l'on cherche à apprendre dans un environnement modifié ou évolutif. Le projet montre donc bien la complémentarité de ces deux familles d'IA.

11. Limites du projet

Notre projet possède encore plusieurs limites. Premièrement, le Q-learning a été entraîné principalement contre un adversaire simple, ce qui ne garantit pas encore une politique optimale face à un adversaire plus fort. Deuxièmement, les résultats comparatifs complets entre Q-learning et MinMax sur toutes les configurations ne sont pas encore stabilisés et doivent être consolidés par une dernière campagne de tests.

Troisièmement, MinMax reste dépendant des paramètres de profondeur et d'heuristique. Son comportement peut donc varier selon le niveau d'optimisation choisi. Enfin, le projet reste volontairement simple sur le plan du jeu afin de rester explicable et maîtrisable dans le cadre d'un projet de cours.

12. Conclusion et ouverture

Ce projet nous a permis de transformer un jeu très simple en un terrain d'expérimentation pertinent pour l'intelligence artificielle. En ajoutant des cases bloquées puis en comparant Q-learning et MinMax, nous avons montré que la structure du jeu influence directement la stratégie, la qualité des décisions et le coût de calcul des algorithmes.

La remarque du professeur sur le risque de match nul lorsque trop de cases sont bloquées a constitué un tournant utile dans notre réflexion. Elle nous a conduits à dépasser l'idée d'un simple morpion modifié pour construire un système plus cohérent, où le blocage et la taille du plateau sont pensés ensemble.

En ouverture, plusieurs pistes sont possibles : prolonger l'entraînement du Q-learning, enrichir la fonction heuristique de MinMax, comparer les deux approches sur davantage de scénarios, ou étendre le travail à d'autres jeux simples. Ces prolongements permettraient d'aller plus loin dans l'étude de la robustesse et de l'adaptation des IA.

13. Annexes

13.1 Répartition du travail

Étudiant	Contribution principale
NKEMLA LIENOU Steve Frede	développement du moteur du jeu, intégration du Q-learning, architecture générale
RICHETER Marie	recherche, structuration théorique, rédaction, optimisation de MinMax

13.2 Éléments à ajouter avant rendu final

- Captures d'écran des modes de jeu ;
- Résultats complets de la comparaison Q-learning / MinMax ;
- Numéro de groupe et date ;
- Éventuelles figures ou schémas explicatifs ;
- Relecture finale et harmonisation du style.

Ajout important : `interface.py`

Dans la dernière évolution du projet, nous avons ajouté le fichier `interface.py`. Ce fichier permet d'afficher une interface graphique plus claire pour la démonstration. Depuis cette interface, il est possible de lancer les différents modes de jeu, d'entraîner l'agent Q-learning et d'afficher les résultats de comparaison entre Q-learning et MinMax.

Annexe complémentaire — Captures de l'interface

Cette partie a été réalisée dans le fichier *interface.py*. Nous voulions ajouter une interface graphique en bonus mais par peur de manquer de temps ou de risquer l'entièreté du programme déjà fonctionnel, nous avons préféré créer un second « main » et ainsi reprendre notre avancement avec une interface graphique développée pour faciliter la démonstration du projet pendant l'exposé. Elles montrent la page d'accueil, les différents modes de jeu, la gestion des cases bloquées ainsi que l'écran de comparaison des deux intelligences artificielles.

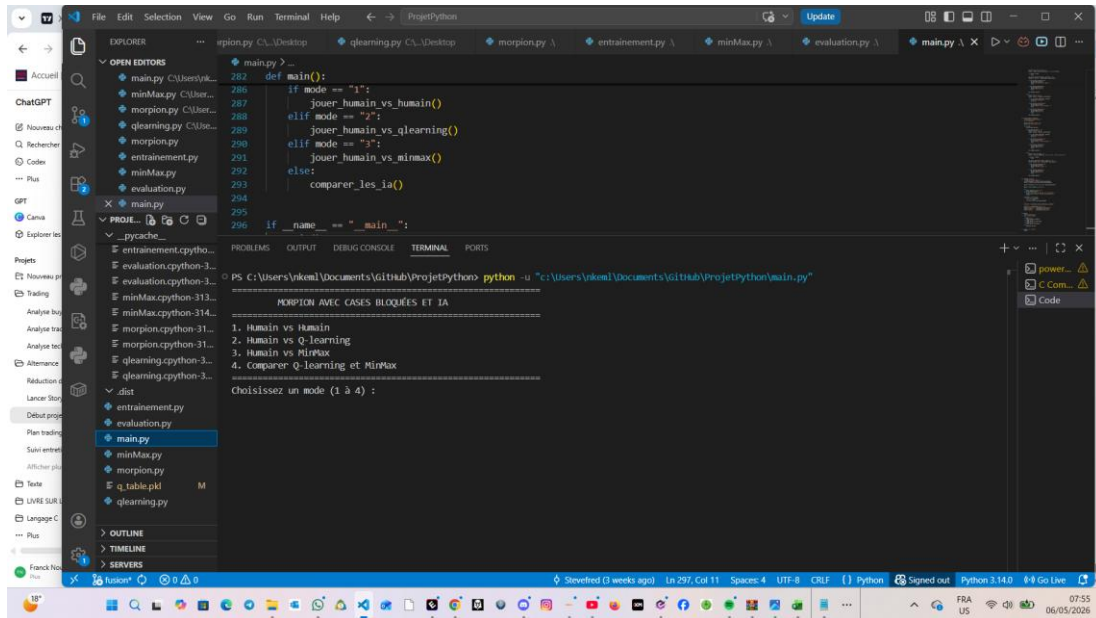


Figure 1 — Page d'accueil de l'interface graphique du projet.

Cette vue montre l'écran d'accueil. L'utilisateur peut directement choisir un mode de jeu, entraîner le Q-learning, comparer les IA ou quitter l'application.

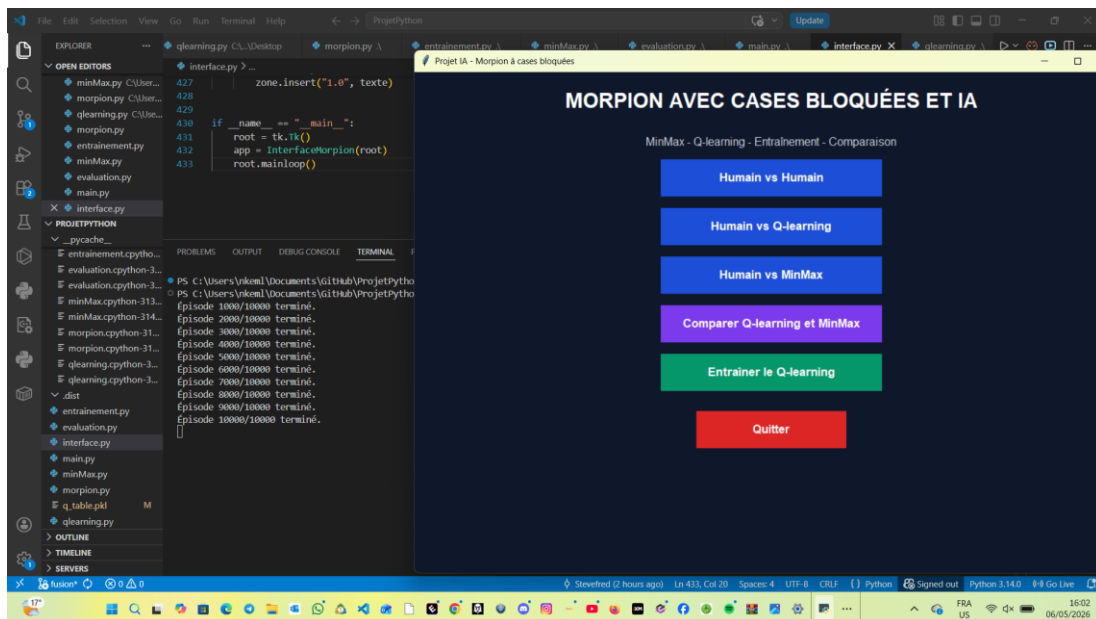


Figure 2 — Mode Humain vs Q-learning : affichage initial du plateau.

Cette vue montre une partie jouée via l'interface graphique. On y observe la grille, les cases bloquées, les symboles joués et le message affiché en fin de partie.

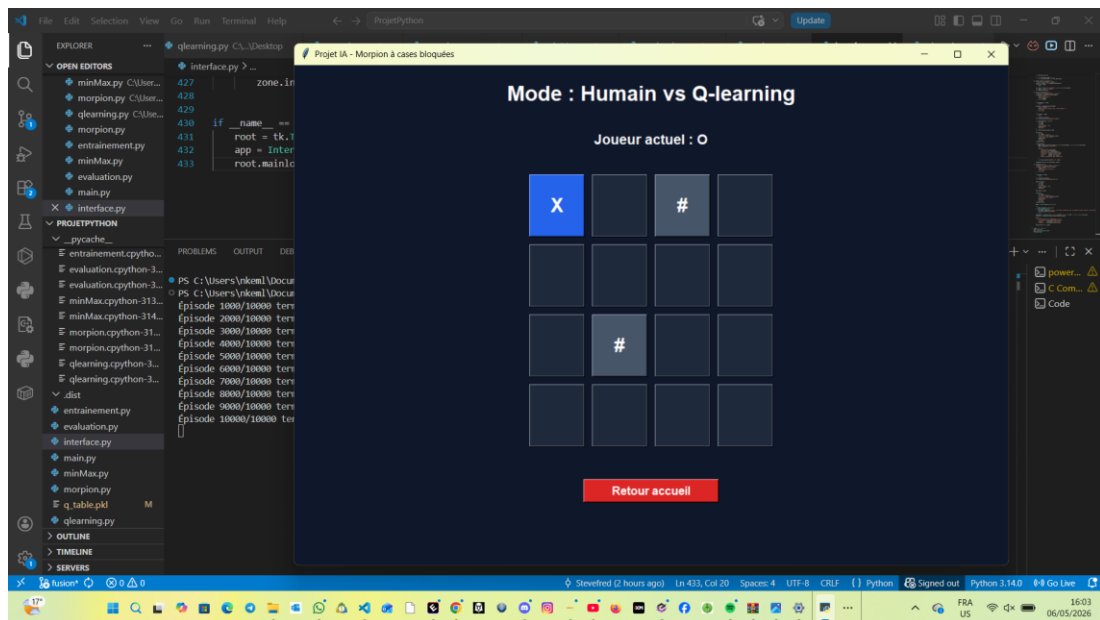


Figure 3 — Mode Humain vs Q-learning : exemple d'une partie se terminant par un match nul.

Cette vue montre une partie jouée via l'interface graphique. On y observe la grille, les cases bloquées, les symboles joués et le message affiché en fin de partie.

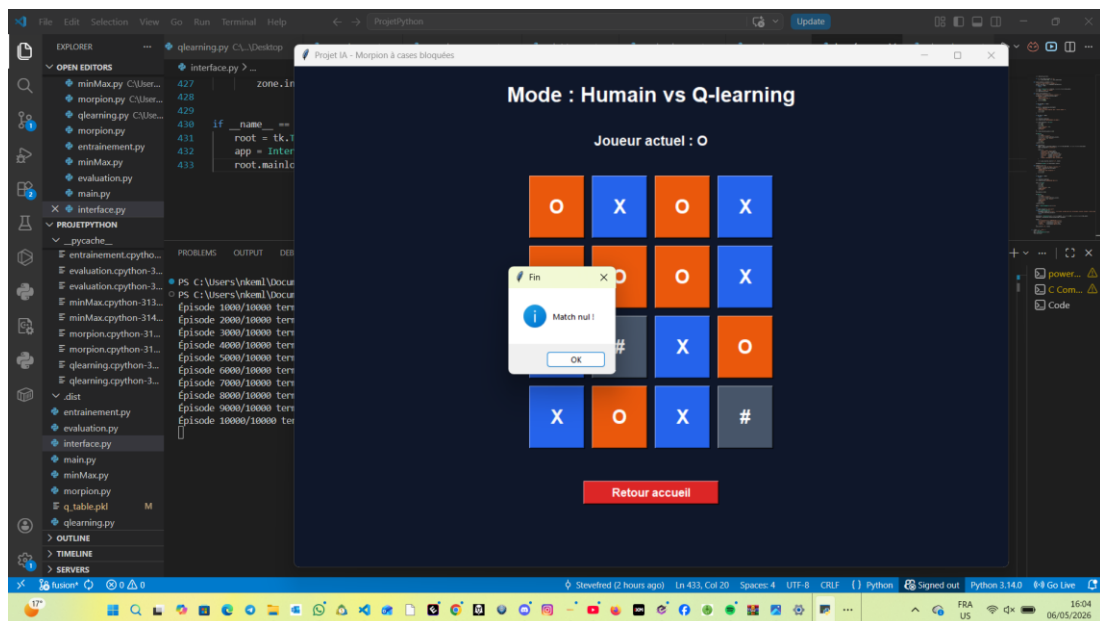


Figure 4 — Mode Humain vs Q-learning : exemple d'une partie gagnée par le joueur O.

Cette vue montre une partie jouée via l'interface graphique. On y observe la grille, les cases bloquées, les symboles joués et le message affiché en fin de partie.

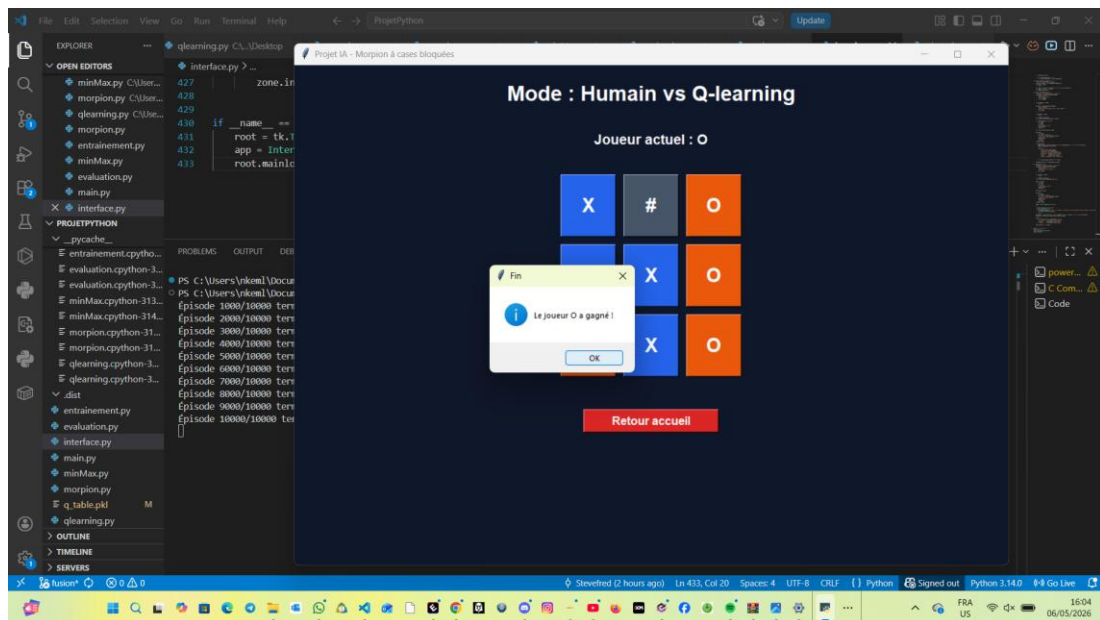


Figure 5 — Mode Humain vs MinMax : exemple de victoire affichée dans l'interface.

Cette vue montre une partie jouée via l'interface graphique. On y observe la grille, les cases bloquées, les symboles joués et le message affiché en fin de partie.

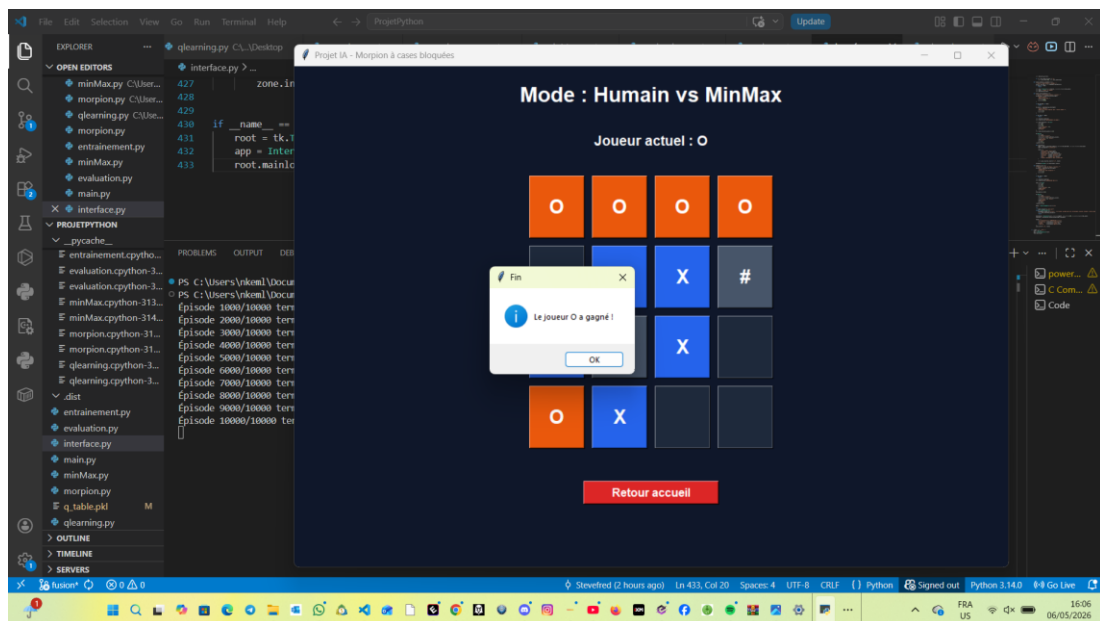


Figure 6 — Écran de comparaison des deux IA dans l'interface graphique.

Cette vue montre l'écran de comparaison. Elle permet d'afficher les résultats obtenus lors du test Q-learning vs MinMax, ce qui facilite l'analyse pendant la présentation.

Arborescence finale du projet

ProjetPython/

├─ morpion.py

├─ qlearning.py

├─ entraînement.py

├─ minMax.py

├─ evaluation.py

- |— main.py
- |— interface.py
- |— q_table.pkl